

18. KNN (K Nearest Neighbors) Classification:-

Q. What is KNN?

- KNN = K-Nearest Neighbors
- It is a supervised learning algorithm.
- Used mostly for classification (can also be used for regression). Non-linear classification.

• Very simple & intuitive method. "look at your closest neighbors".

2. How KNN works (concept)

1. You have a dataset with labels (e.g. types of Iris flowers).

2. When a new point comes, you:

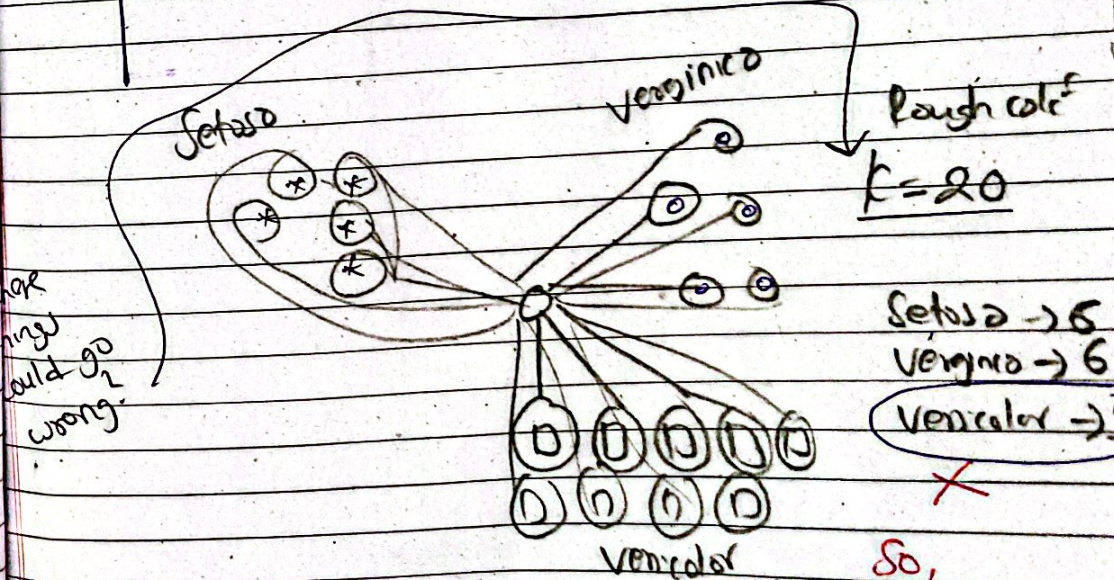
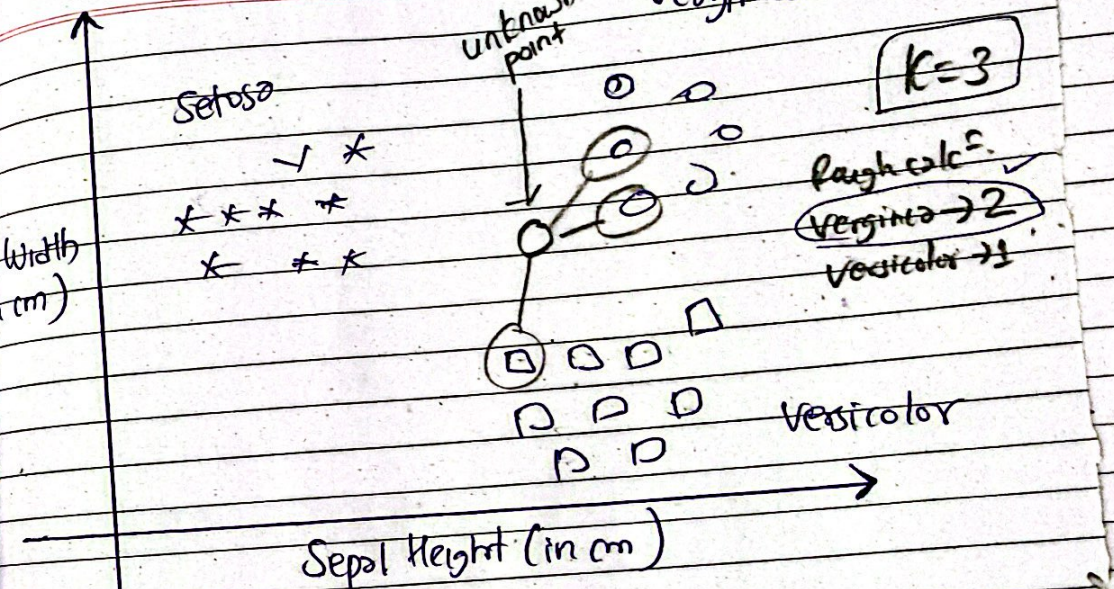
• choose a value of k (examples: $k=3$ or 5)

• find the k nearest points in the training data

• Use Euclidean distance to measure closeness.

3. Majority vote among those k neighbors determines the class.

Ex - New point is yellow.
 $k=3$ to check 3 nearest neighbors.
 If neighbors are: Virginica, Virginica, Virginica
 Predict = Virginica.



Also, check for imbalanced classes data.

3. Choosing k

- No fixed rule for choosing k
- Usually small numbers like 3 or 5.
- If:
 - k too small noisy, overfitting
 - k too large $\rightarrow$ may mix other classes, misclassification

Example:-

- $k=20$ may give wrong result if class distribution is unbalanced.

4. KNN with more features

- In example, only 2 features: sepal width & sepal height.
- KNN works fine even with n features (multidimensional space).
- We can't visualize them, but math still works.

5. Iris Dataset

- comes from sklearn-datasets
- has 4 features
 - 1 - sepal length
 - 2 - sepal width
 - 3 - petal length
 - 4 - petal width

• classes:

- 0 - setosa
- 1 - versicolor
- 2 - virginica

Total samples = 150
(50 per class)

6. Train/Test Split

• Usual practice - split into training & testing

• Example:-

- Training samples = 120
- Test samples = 30

~~Imp~~

7. Building a KNN model (Python steps)

1. load dataset

```
from sklearn.datasets import load_iris  
iris = load_iris()
```

2. Make DataFrame

- add features + Target column

3. Split into train/test

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(...)
```

Imp

4. Create KNN classifier

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

5. fit the model

```
model.fit(X_train, y_train)
```

6. Calculate accuracy

```
model.score(X_test, y_test)
```

8. Euclidean Distance (Default Metric)

• In sklearn, `metric = "minkowski"`, `p=2`

=> This means Euclidean distance

9. Confusion Matrix

- tells which classes were predicted correctly or incorrectly.

Steps:

1. predict on test data:

```
y_pred = model.predict(X_test)
```

2. Create confusion matrix:

from sklearn.metrics import confusion_matrix
cm = confusion_matrix (y_test, y_pred)

3- Heatmap (if using seaborn):

- Diagonal values = correct predictions.
- Off-diagonal = wrong predictions

Example meaning:-

- Value at

10- Classification Report

- Shows:
 - Precision
 - Recall
 - f1-score for each class

Imp (off-diagonal
confusion matrix)

from sklearn.metrics import classification_report
print (classification_report (y_test, y_pred))

11- Important Concepts

- Accuracy
- Precision & Recall.

Choosing best value of K

You can find the best K using:-

- Trial & error (manual)
- GridSearch CV
- k -fold Cross Validation

*Short Summary (Easy to Renew)

- k NN = simple, intuitive classification method.
- find k nearest neighbors $\rightarrow$ take majority class
- choose k carefully.
- Uses Euclidean distance by default.
- Easy to implement in sklearn.
- Evaluate using accuracy, confusion matrix, classification report.